

Predicción numérica: Regresión, Ridge y Lasso

Jorge Gallego

Inter-American Development Bank

Bogotá Summer School in Economics — 2026

Agenda de la sesión

- Cambiamos de problema: del **clasificar** (categorías) al **predecir valores numéricos**
1. Regresión lineal y logística (un repaso con mirada predictiva)
 2. Regularización: Ridge y Lasso
- La regresión es, también, un algoritmo de *machine learning*: estima parámetros con los datos para hacer predicciones

Parte 1

Regresión lineal y logística

Introducción

- Hasta ahora solo hicimos predicciones **categorías** (clasificación)
- En muchos casos lo que importa es una predicción **numérica**
- La **regresión** es, de lejos, el método más usado para esto
- Haremos un repaso rápido (seguramente ya la dominan), pero con énfasis en el enfoque **predictivo**

Fundamentos básicos

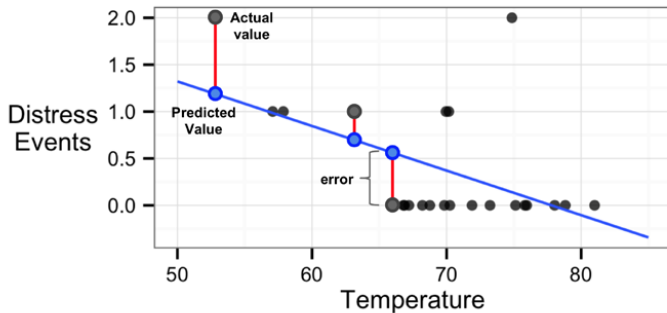
- Una regresión especifica la relación entre una variable **dependiente** y y una o más **independientes** x
- El método se usa con tres enfoques distintos:
 - ▶ describir cómo *varían* las características observables
 - ▶ cuantificar relaciones *causales*
 - ▶ identificar patrones para *predecir* comportamiento futuro
- Nosotros enfatizaremos el enfoque **predictivo**

Regresión lineal simple

- y depende de un único predictor x de forma lineal:
 $y = \alpha + \beta x + \varepsilon$
- Estimamos α y β por **Mínimos Cuadrados Ordinarios** (OLS): minimizando la suma de residuos al cuadrado

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n e_i^2$$

Regresión lineal simple



Correlación

- La correlación mide la asociación **lineal** entre dos variables
- El indicador más usado es el coeficiente de Pearson:

$$\rho_{x,y} = \frac{\text{Cov}(x,y)}{\sigma_x \sigma_y}$$

- Va de -1 a 1 ; cuanto más se aleje de 0 , mayor asociación (débil: $0.1-0.3$; moderada: $0.3-0.5$; fuerte: > 0.5)
- Ojo: correlación **no** es causalidad

Regresión lineal múltiple

- Extendemos a varios predictores:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_k x_k + \varepsilon$$

- En forma matricial, $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \varepsilon$, y la solución OLS es:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- Con $\hat{\boldsymbol{\beta}}$ predecimos: $\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\beta}}$. Es un algoritmo más de *machine learning*

Ventajas y desventajas

- **Ventajas:** es el método más popular para datos numéricos; se adapta a casi cualquier tarea; estima la fuerza y el tamaño de cada relación
- **Desventajas:** hace supuestos fuertes sobre los datos; la especificación se decide *ex ante* (el humano elige las variables); no maneja por sí solo los datos ausentes

Variable dependiente dicotómica

- ¿Y si el *outcome* es una **dummy** (0/1)? Por ejemplo, “¿el cliente hace default?”
- Podríamos usar OLS directamente: el **Modelo de Probabilidad Lineal** (MPL)

$$y = \beta_1 + \beta_2 x_2 + \cdots + \beta_k x_k + u$$

- Es simple, pero tiene problemas...

Problemas del MPL

1. Nada garantiza que la probabilidad predicha esté entre 0 y 1 (¡puede dar negativa o mayor que 1!)
 2. El efecto marginal de cada x es **constante** (poco realista)
 3. Se viola la homocedasticidad (esto es lo menos grave: se corrige con errores estándar robustos)
- Los modelos **no lineales** (Logit, Probit) resuelven los dos primeros

Modelos no lineales

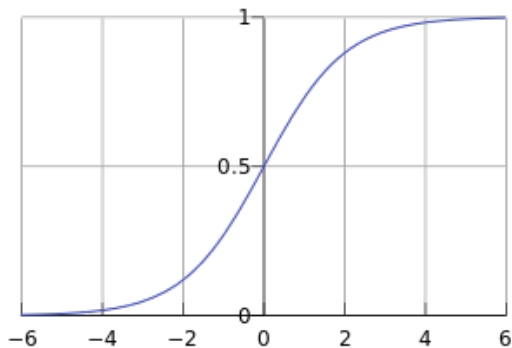
- Modelamos la probabilidad de éxito con una función G que siempre cae entre 0 y 1:

$$P(y = 1 \mid \mathbf{x}) = G(\beta_1 + \beta_2 x_2 + \cdots + \beta_k x_k) = G(\mathbf{x}\boldsymbol{\beta})$$

con $0 < G(z) < 1$ para todo z .

- **Logit:** G es la función logística, $\Lambda(z) = \frac{e^z}{1 + e^z}$
- **Probit:** G es la normal estándar acumulada, $\Phi(z)$

La función logística (Logit)



Estimación: máxima verosimilitud

- En el MPL se usa OLS; en Logit y Probit **no** se puede
- Se usa **máxima verosimilitud** (EMV)

Intuición

Se eligen los parámetros $\hat{\beta}$ que hacen más probable observar los datos que efectivamente tenemos: si $y_i = 1$, el modelo debería predecir una probabilidad cercana a 1 para ese caso.

Bondad de ajuste

- **Porcentaje de pronósticos correctos:** si $\hat{P}(\mathbf{x}_i) > 0.5$ predecimos $\hat{y}_i = 1$, si no $\hat{y}_i = 0$; medimos la proporción de aciertos
- **Pseudo R^2 :** $1 - \frac{L_{irr}}{L_0}$, comparando la verosimilitud del modelo completo con la del modelo solo con intercepto
- **Al taller:** predecir gastos médicos con regresión (lineal y logística) sobre la base `insurance.csv`

Parte 2

Regularización: Ridge y Lasso

¿Por qué ir más allá de OLS?

- El modelo lineal es muy útil, pero para **predecir** tiene dos problemas
- **Poder predictivo**: si el número de variables p es grande frente a las observaciones n , la varianza de OLS crece (y si $p > n$, ¡no hay solución única!)
- **Interpretabilidad**: con muchas variables irrelevantes, el modelo es difícil de leer, y OLS casi nunca da coeficientes **exactamente 0**

Regularización

- Los métodos de **regularización** atacan ambos problemas
- Idea: “encoger” (*shrink*) los coeficientes hacia 0 para las variables que lo merecen
- Según el método, los coeficientes se acercan a 0 o se vuelven **exactamente** 0 (selección de variables)
- Esto reduce la varianza y mejora la predicción (a costa de un poco más de sesgo)

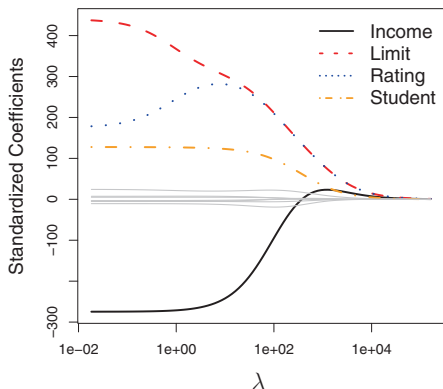
Regresión Ridge

- OLS minimiza la suma de residuos al cuadrado (RSS). Ridge añade una **penalidad**:

$$\underbrace{\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2}_{RSS} + \lambda \sum_{j=1}^p \beta_j^2$$

- $\lambda \geq 0$ es un parámetro de **sintonización**: pesa cuánto importa la penalidad
- Si $\lambda = 0$, Ridge = OLS; si $\lambda \rightarrow \infty$, los coeficientes tienden a 0

Ridge: trayectoria de coeficientes

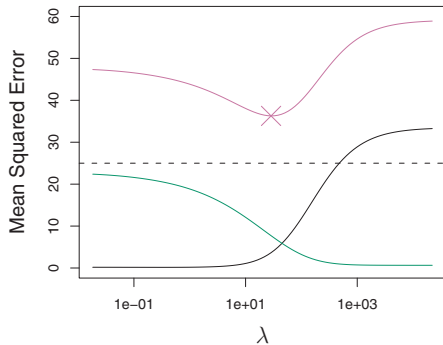


A mayor λ , más se encogen los coeficientes hacia 0

Ridge vs. OLS: el trade-off sesgo–varianza

- ¿Por qué Ridge puede mejorar a OLS? Por el **trade-off sesgo–varianza**
- Al subir λ : baja la **varianza** de las predicciones, pero sube el **sesgo**
- Existe un λ que minimiza el error cuadrático medio ($\text{MSE} = \text{varianza} + \text{sesgo}^2$)
- Ridge ayuda mucho cuando p es grande frente a n

Ridge vs. OLS



Verde: varianza Negro: sesgo² Morado: MSE

Lasso

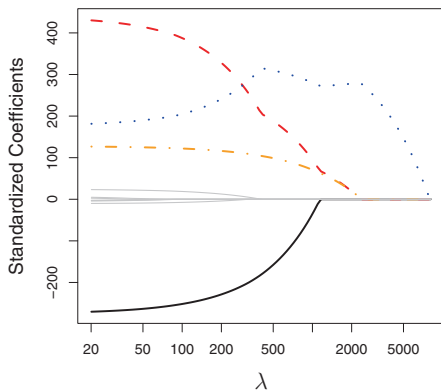
- Problema de Ridge: los coeficientes nunca son **exactamente** 0, así que ninguna variable se descarta del todo
- **Lasso** cambia la penalidad de β_j^2 a $|\beta_j|$:

$$RSS + \lambda \sum_{j=1}^p |\beta_j|$$

- Igual que Ridge, encoge los coeficientes; pero **a diferencia** de Ridge, algunos sí se vuelven 0

Lasso: selección de variables

- Al volver 0 algunos coeficientes, Lasso **selecciona variables** automáticamente
- El modelo resultante es más fácil de interpretar



¿Lasso o Ridge?

- Ninguno domina al otro en general
- Si el *outcome* depende de **pocas** variables, suele ganar **Lasso**
- Si depende de **muchas** variables, suele ganar **Ridge**
- En la práctica, dejamos que los datos decidan, vía **validación cruzada**

La selección de λ (abrebocas)

- ¿Cómo elegir λ ? La intuición es simple:
 - ▶ se prueba un conjunto de valores de λ
 - ▶ para cada uno se calcula el error de predicción del modelo
 - ▶ se elige el λ con el mejor error
- La clave está en **cómo** se construye ese error: **validación cruzada** (*cross-validation*)
- Lo veremos en detalle en la próxima sesión, sobre **desempeño de modelos**

Resumen de la sesión

- La **regresión** predice valores numéricos; el **logit/probit**, probabilidades para un *outcome* binario
- **Ridge** y **Lasso** regularizan: encogen los coeficientes para mejorar la predicción cuando hay muchas variables
- **Lasso** además hace selección de variables (coeficientes exactamente 0)
- El parámetro λ se elige con **validación cruzada**: el puente hacia la próxima sesión

Más allá de los modelos lineales

- Para predecir un **número** no estamos limitados a modelos lineales (regresión, Ridge, Lasso)
- **Árboles de regresión**: dividen los datos en grupos y predicen un valor por hoja; capturan relaciones no lineales
- **Model trees**: árboles que ajustan una **regresión** en cada hoja
- **Ensamblados y boosting** (random forests, **XGBoost**): combinan muchos árboles y suelen dar la mejor predicción

Árboles de regresión

- Misma idea que los árboles de clasificación (sesión 2), pero ahora el objetivo es un **número**
- Parten los datos con reglas “si... entonces...”; en cada **hoja** predicen el **promedio** del *outcome* de los casos que caen ahí
- En vez de minimizar la impureza (Gini), eligen los cortes que más reducen la **varianza** (el error cuadrático) dentro de cada hoja
- **Ventajas:** capturan no linealidades e interacciones, son interpretables y no requieren escalar las variables
- **Límite:** la predicción es “escalonada” (constante por región) y un solo árbol sobreajusta con facilidad

Model trees

- Un refinamiento del árbol de regresión
- En lugar de predecir un **promedio** en cada hoja, ajusta una **regresión lineal** dentro de cada hoja
- Combina lo mejor de dos mundos: la partición **no lineal** del árbol y la **suavidad** de la regresión local
- Suele predecir mejor que un árbol de regresión simple
- En R, el algoritmo clásico es **M5** (lo vemos en el apéndice de *regression trees*)

Random forests para predicción numérica

- Un solo árbol tiene mucha **varianza** (cambia bastante según la muestra). La solución: promediar **muchos** árboles
- Cada árbol se entrena con una muestra *bootstrap* distinta y un subconjunto aleatorio de variables (igual que en clasificación, sesión 2)
- La predicción final es el **promedio** de todos los árboles (en clasificación era el **voto** mayoritario)
- Reduce el sobreajuste y suele dar el **menor error** de predicción
- **Costo**: pierde interpretabilidad, pero entrega una medida de **importancia** de cada variable

¿Cuál elegir, y dónde se profundiza?

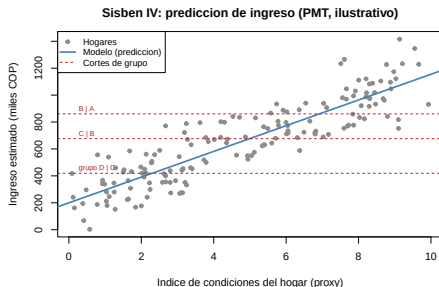
- Regresión / Ridge / Lasso: **transparentes e interpretables**. Árboles y ensambles: **más flexibles y precisos**, pero menos interpretables
- El *trade-off* central: **simplicidad** vs. **flexibilidad**; pesa más cuando hay que **explicar** la decisión
- **Dónde se profundiza:** árboles de regresión y *model trees* en el **apéndice**; ensambles y *boosting* en la **sesión 4** (árboles y random forests ya aparecieron en la **sesión 2**)

Aplicación real en Colombia: Sisbén IV

- ¿Cómo decide el Estado **quién recibe** subsidios y programas sociales? Necesita medir la capacidad económica de cada hogar
- El **Sisbén** (administrado por el DNP) clasifica a los hogares para **focalizar** el gasto social
- El reto: estimar el **ingreso** de un hogar a partir de variables observables (vivienda, educación, activos)

Sisbén IV: los métodos

- El **Proxy Means Test** es **predicción numérica**: estimar el ingreso a partir de muchas variables — el problema de esta sesión
- Con **300+** predictores, la **regularización (Ridge/Lasso)** es la herramienta natural: evita sobreajuste y selecciona variables
- Sisbén IV usa una variante más flexible (**gradient boosting**): mismo problema, modelo más complejo



Sisbén IV: resultados y lección

- Clasifica a los hogares en grupos (A, B, C, D) que ordenan el acceso a los programas
- Es la **puerta de entrada** a buena parte de la política social del país
- Como el modelo decide **quién recibe ayuda**, la **transparencia** importa tanto como la precisión: por eso elegir entre regresión (interpretable) y boosting (flexible) no es trivial
- *Fuente:* DNP — Sisbén IV; modelo de presunción de ingresos (*Proxy Means Test*).